SCORE: Synthesising Complete, Owned, Responsive Environments for Simulation

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 27 October 2026

Abstract

Games, film, robotics and embodied AI all need 3D environments in which every object is a physical body of its own, that can still be edited after they are made, and that belong to whoever made them. World models produce video that no simulator can open, and one-shot 3D generators produce a single fused scene. SCORE is an open framework that synthesises such environments the way film and game studios build them: in OpenUSD, scene within scene, with layout, surfaces, physics, lighting, motion, effects, sound and people each held in a layer of its own. A creator locks the design from the top down. Open vision models, procedural tools and coding agents produce every layer beneath it, and each output must pass measured acceptance tests before the next is made. The result is a single OpenUSD environment whose physics a simulator reads as written, and every person in it also receives a musculoskeletal body that follows its motion in simulation. This opens a route to the training data that recorded measurement cannot supply at scale: movement in complete scenes, with physics-derived ground truth down to the forces of individual muscles. Code and paper: https://github.com/Babon-Innovations-b-v/score (MIT licence).

Gap: Figure 1, one environment, every layer. Environment 1 from the top down: its physics (the bodies it stands on and their gravity), a top scene on its site, a parent scene with its child scenes, and one scene opened into its layers side by side (base, settle, physics, light, motion, effects, sound, people). It waits on environment 1's batch.

1 Introduction

Simulated 3D environments serve games, film, robot learning and embodied AI [1]. Across these uses they share a common structure of separate objects with collision geometry, mass and joints, a ground surface with its gravity, light and sound emitted by identifiable sources, and scenes that can be reopened, modified and simulated again. Authoring such an environment by hand is expensive, and a generative system that takes over this work must leave the result in the hands of its author, since in the Game Developers Conference's 2026 survey 52 percent of game workers considered generative AI to be harming the industry [2].

World models generate an environment as a sequence of video frames [3], [4] that no physics simulator can step forward in time. 3D environment generators [5], [6] produce geometry, mostly as a single fused asset from which separate objects have to be recovered afterwards [7], [8]. Both operate in a single pass, so a fault in the output cannot be traced to the step that caused it. Procedural content generation systems [9], [10] and coding agents, language models that write scenes as executable programs [11], [12], [13], [14], [15], [16], keep the environment explicit and inspectable, but they either reconstruct a scene that already exists or assemble it from a fixed asset library. The models

they produce often float above their supports or come apart [17], such agents assess their own geometry unreliably [18], and on scenes reconstructed from video the strongest model scores 0.91 on static scenes but only 0.67 on dynamic ones [19]. Open vision foundation models [20], large models pretrained on broad image data and reused across tasks, now recover 3D shape [21], segmentation masks [22] and depth [23] from images, and a coding agent can call them as tools, yet none of them gives a consistent visual style across objects that different models produced [24], [25], and none gives physical properties.

The components needed for the physics already exist. OpenUSD [26], the scene description that Pixar developed for its film production, stores each department's work in its own layer file and composes scenes from smaller ones by reference, and its physics schema describes rigid bodies, colliders, mass and joints within the same stage, the composed scene. MuJoCo [27] is a widely used physics engine for articulated bodies in contact, and Newton [28], built on the Warp GPU programming framework [29], runs MuJoCo's solver on the GPU [30] and loads OpenUSD scenes directly. What is missing is a route from what a creator intends to a complete environment in this form.

SCORE is that framework. Its environments are complete, since every object, surface, light and sound a scene shows exists wherever the viewer looks. They are owned, because every model in the framework permits commercial use of its output, so the creator holds the full rights. They are responsive, because every object carries its collider, mass and material and every moving part its joints, so a simulator steps the environment as written. They are editable, since each scene is a stack of layers produced by steps that are each identified by a key derived from their inputs, and a change reruns only the steps it affects. An environment can range from a house on a plot of ground to a system of worlds that move one another by gravitation.

We contribute the SCORE framework, which turns a creator's locked design into a single OpenUSD environment that is physically simulable and opens unchanged in a game engine, a robot simulator or a film renderer. Its method (Section 2) builds a scene from the top down and layer by layer, with each stage passing measured acceptance tests before the next begins. The evidence plan (Section 3) comprises four demonstrations on a first environment, left as gaps until its batch has run. Section 4 sets out where the framework goes next, towards simulated people whose bodies are true to the individual person.

2 Method: building an environment

2.1 From concept to plan

The environment concept. The creator first locks the EC, which fixes the environment's visual style and physical setting together with his reference images and the scenes it contains, in a short structured interview conducted with the vision-language model Gemini 3.1 Pro [31]. An environment plan then places the site of each top scene on that ground and computes the sky visible from it.

A concept for each scene, top down. A concept image is then locked for each scene, parents before their child scenes. Each candidate image is one call to the image generation model Nano Banana Pro [32], made on a concept review page where the creator marks regions and requests a new image. A child scene is drawn only once its parent has been locked and planned, from a crop of the parent's concept, so that it fits the parent it stands in.

Inventory and plan. Gemini 3.1 Pro lists each object of the locked concept as a row of the scene

inventory, giving its kind, count, anchor and size in metres, and the segmentation model SAM 3 [22] localises each row with a bounding box. Nothing outside the inventory is generated or placed. Nano Banana Pro then draws the scene from directly above, and Gemini 3.1 Pro writes the plan in metres from that drawing, the concept and the inventory. The plan gives the outline, walls, openings, levels, stairs, roofs and terrain, each item with its orientation, people as the spots on which they stand, and the rooms of the child scenes. Free checks, which make no paid model call, test that every row is placed as many times as its count, inside the outline and clear of other floor items, that every door and standing spot can be reached on foot, and that the child scenes tile their parent. Any remaining violations are returned to the model as notes at most twice, and only what still fails is recorded as a gap. The creator's edits on the plan review page always take precedence and cause everything that follows to be recomputed.

Shapes first. The route follows one coarse-to-fine loop at every scale. A parent scene receives its concept, plan and a rough block model, and its plan is divided into the shapes of its child scenes. Once those shapes are fixed, all child scenes are produced in parallel, each repeating the loop. The stages `rough-model` and `shell-split` are implemented but have not yet run.

2.2 Layers from row kinds

A scene's layers are derived from its design alone. Each inventory row's kind, determined from the noun in the inventory reading by a single table of rules, states what its objects do, whether they emit, move, lie loose, glow, grow, act or sound. A scene receives an effects, motion, physics, crops or people layer only for rows that do so, and always a light and a sound layer. Each layer is generated from its rows by the rule of their kind, so a new environment needs no hand-written list of layers. The scene record holds what no generated object covers (building shells, stairs, ground and water, the distant backdrop, lamps and the neighbouring scenes its doorways lead to), and its gate requires the shell to lie within 5 cm of the plan and every doorway to open onto a floor.

Models. Objects follow the prop route first, which turns a single image of an object into a 3D model. Nano Banana Pro draws a close-up of the object from its crop of the concept and redraws it at most three times when the shape check rejects it. The image-to-3D model Pixal3D [21] then reconstructs the mesh, and a likeness gate compares its silhouettes from eight viewpoints with the close-up. Only an object whose checks fail is built instead by a procedural builder, a program that a coding agent writes, and a thin straight object (a pipe, post or beam) whose every close-up failed is generated in code by sweeping its cross-section along its length.

Surfaces. Everything a scene shows comes from the creator's asset library, score_lib, or is recorded as a written exception. A generated model contributes geometry only, and each of its parts takes one material from a library of procedural materials, written as ProcFunc functions [33] on Infinigen's shaders [9] and coloured from the scene's palette. The finish route labels each face of the model from free renders, matches each finish to a library material, bakes it into textures and checks the bake against the close-up, requiring each finish to lie within a colour difference of ΔE 32 of its colour there. A free library check traces every material, model, light, sound and character of an accepted assembly back to the library or to an exception.

Settle and physics. Loose objects are dropped onto the generated ground in a rigid-body simulation, the settle, and an object that would rotate by more than 15 degrees or drift by more than 30 cm is recorded as a resting fault and keeps its laid-out pose. The settle writes the pose in which each object comes to rest as a layer. The physics trial then steps the composed scene in Newton, using MuJoCo's solver with Newton's contact handling under the gravity of the body the scene

stands on, and evaluates it with the gates `reacted`, `at rest`, `energy` (no increase in the energy of undriven bodies) and `hinge` (a hinged door leaf swings to its joint limit). The trial moved from Blender's Bullet engine [34] to Newton on 27 October 2026. On a test scene at lunar gravity (two dropped crates, a driven door with a wheel fixed to it and a pushed stool), both engines gave the same result on every gate, and Newton brought the crates to rest on their targets where Bullet left them standing off by its contact margin of 8 mm. Newton's deterministic mode, which reproduced bitwise-identical results over six runs of that test scene, cannot handle scenes with many loose bodies, because its buffers exceed the 32-bit limit, so the trial and the settle run without it. Two runs of a 25-body test scene then agree within 5e-7 m for every body except one that falls off the edge of the floor. The settle moves to Newton in the same way, so that one engine steps both what comes to rest and what is tested, and one effect is simulated in Newton as a first case for the effects layer.

Light, motion, effects and sound. The sky is computed from the physical setting in the EC, never drawn, with each celestial body at its true direction, apparent size and phase as seen from the scene's site. The lamps are the inventory's glowing rows. One render per scene places each lamp in its own light group, and a free sweep mixes their intensities to bring the scene's brightness within a factor of 1.5 of its concept's. Every object of a moving row moves by the rule of its kind at a speed taken from cited real machines, and is gated on nothing coming within 3 cm of anything else. Emitters are measured against real footage. Each sound source receives a library file or one take generated by the text-to-audio model MOSS-SoundEffect [35], gated on an audio-text similarity score from CLAP [36] of at least 0.30.

People. People are models like any other, each made from a close-up per person row, reconstructed by Pixal3D and given its materials by the finish route. The SOMA parametric body model [37], estimated from the close-up by SAM 3D Body [38], is fitted beneath the model as its skeletal rig and animated with motion clips from Kimodo [39]. A shell gate accepts a clothed appearance when 99 percent of the points of its non-rigid parts lie within 5 cm of the body. Each person stands on a person spot of its scene's plan.

Sim bodies. Each character also receives a sim body, its physical twin in Newton, so that the people in an environment are physically simulated as well as animated. The minimal body is an articulated rigid-body model built directly from SOMA's 23 segments, with a ball joint at each SOMA joint frame. It is driven by the character's SOMA segment rotations without scaling or per-person setup, its mass is distributed according to its segment volumes, and it stands on a SCORE floor so that its ground reaction forces arise from contact. Muscles are added as a minimal first step. Built this way, the body follows SOMA's rotations to within 6.9e-9 m, and in Newton it matches plain MuJoCo to within 3.4e-8 after 200 steps.

Gap: Newton for the settle, one effect and the sim body. The settle and the physics trial on one engine, with the resting and physics gates per scene against the Bullet settle; the one effect stepped on Newton; and a sim body walking on a SCORE floor, its joint angles and ground forces against OpenCap's LabValidation motion capture and force plates [40]. It waits on these three stages being built and run on a cloud card.

Completion. A scene is complete when its completion gate passes on its final assembly, with every inventory row made or dropped with a reason, nothing left unmade, no resting fault, and every layer and every visible scene present. A fault the creator chooses to ship with is an accepted gap, recorded with who decided, when and why. The gate lists every gap by name and is never relaxed. A hook prevents a coding agent from claiming that a scene is done while its gate fails, because agents judging their own scenes agree with a measured geometry score only 45.8 percent of the time [18]. The films of a complete scene are rendered from scripts the creator has locked.

3 Evidence: environment 1

Environment 1 serves as the evidence. The creator did not accept its first run, and its store of outputs was lost on 19 October 2026, so this draft shows none of its outputs, and each demonstration below will be produced by its new batch. Figure 1 is the first.

Gap: Demonstration 2, physics that holds. Environment 1's scenes stepped on Newton under their own body's gravity: loose objects come to rest, doors swing on their hinges, with the physics trial's gates per scene and Bullet run on the same scenes as the baseline. It waits on environment 1's batch and on Newton taking over the settle.

Gap: Demonstration 3, one environment, many uses. The same OpenUSD environment opened unchanged in a game engine, in a robot simulator and in a film render, with what each reads of its layers and what it drops. It waits on environment 1's batch and on the engine adapter.

Gap: Demonstration 4, an edit that stays cheap. The creator changes one thing in a locked scene, and only the stages that the change affects run again; the stages rerun, time and euros against a full rebuild. It waits on environment 1's batch and on the edit layer being carried from one assembly to the next.

Gap: Table 1. Per scene of environment 1: inventory rows, models generated and code-built, layers, gaps before and after the batch, time, euros and the creator's verdict. It waits on environment 1's batch and the creator's review.

Disclosure. The creator's locks, picks, accepts and waivers are his own actions in the cockpit, and everything between them is automatic. Coding agents wrote most of the framework's code under the author's direction, and 2,413 of its 2,436 commits list a coding agent as co-author. Manual fixes to a scene are recorded as the creator's edits.

4 Future work: people true to the person

The current sim body is a starting point. This section describes goals, not results.

Bodies true to the person. The sim body is to become a musculoskeletal model of the individual person rather than a scaled generic template, including older people and people with a curved spine. MyoSuite's MyoFullBody [41], a full-body model with 416 muscles whose arm and hand derive from the MoBL-ARMS model [42], [43], is the starting point. Each segment of the SOMA body is fitted to the group of bones that spans the same part of the body, from the same joint to the same joint, so that the joints within a group, such as the vertebrae of the spine, share its rotation within their ranges of motion. Virtual markers on bony landmarks of the SOMA mesh add the shape of this person's skeleton, and segment masses are derived from the volumes of the SOMA body. Every input that SOMA does not provide is marked as assumed and given a plausible range. MyoFullBody already runs in Newton, but it still drifts 1.5 cm from plain MuJoCo after 300 steps, so Newton must first be validated for such bodies. The joint angles, and then the net joint moments and ground reaction forces, are to be validated against OpenCap's LabValidation data.

Scenes from motion. Motion comes from the SOMA ecosystem: estimated from video with GEM-X [44], or generated from text and constraints with Kimodo [39], in practically unlimited quantity. The terrain a movement implies becomes a SCORE scene, so the movement keeps its setting without the source video or anything that identifies a person. The sim body follows each movement on the scene's own ground. For every frame, the simulation solves for the ground reaction forces from contact, the net joint moments, and the activations and forces of the muscles that reproduce the

movement. Whatever force and moment neither the ground nor the muscles can supply remains at the pelvis as the residual. The residual is recorded and never applied to the body, so it measures how dynamically consistent the movement is, and only movements with a small residual are kept. Biomechanics uses the same residual to assess recorded motion [45], [46], and PhysibleMDM [47] minimises an inverse-dynamics residual to make generated motion feasible for a torque-driven robot. We apply the criterion to generated motion in a muscle-driven body.

Layered training sets. The same simulations yield training sets in layers. Each sample pairs a rendered image of a clothed person in a complete scene with ground truth for every layer beneath it: the SOMA body and its joint angles, the ground reaction force under each foot, the muscle activations and forces, the residual, and the depth, segmentation and layout of the scene. Their main use is to train estimators that infer muscle activations, joint moments and ground reaction forces directly from ordinary video, fast enough for routine clinical use. Recorded data cannot supply this at scale: muscle forces cannot be measured without invasive methods, and force plates and electromyography confine measurement to a laboratory. Simulation can generate the bodies, movements and settings an estimator must cover, including those of a specific patient group. BioHuman10M [48] already pairs video, part of it rendered, with the activations of a 112-muscle model and with ground reaction forces predicted by a learned model. Our sets would add scenes generated for the purpose under varied lighting, terrain and clothing, ground reaction forces from contact with each scene's own ground, a 416-muscle model, and a residual for every sample. Their value depends on whether estimators trained on them transfer to real force-plate and electromyography recordings.

Brains. A brain is a trained controller that drives a sim body's muscles. Trained on the retained movements and the muscle forces that produce them, it is placed in SCORE's synthesised worlds under their own gravity and terrain, where it must adapt to situations no recording contains. Every run it drives adds movement to the training sets, so it extends the data beyond what a motion generator produces. Muscle-driven controllers such as KINESIS [49] and MuscleMimic [50] are trained on motion capture, as are torque-driven ones such as PHC [51]. To our knowledge, no muscle-driven controller has yet been trained on generated motion selected by its residual. That claim holds only once such a controller is compared with one trained on motion capture and with one trained on the same generated motion unfiltered.

The data loop. The amount of data scales with the compute put behind it, since every scene, movement and run adds samples. This is why SCORE is open: others can apply their own compute to the same framework and share the resulting datasets. Estimators specialised for clinical use would be trained outside SCORE, on data that never enters it.

5 Limitations and conclusion

A single creator judges the results, and he has not yet accepted a run. Closed models (Nano Banana Pro, Gemini 3.1 Pro) draw the concepts and close-ups and read and plan every scene, and may change or be withdrawn. The design step, the layers derived from row kinds, the finish route, the shapes-first loop and the scene tree have not yet carried a scene through a batch. The edit layer is so far only a reserved slot. The settle, the effects and the sim body do not yet run in Newton as stages, and no game engine or robot simulator has yet opened the result. Next come environment 1's batch with its four demonstrations, an engine adapter, the edit layer carried across assemblies, further environments, open models replacing the closed ones, and the data loop, in which residual-filtered generated movement trains muscle-driven controllers whose value a comparison with motion

capture has yet to show. SCORE synthesises an environment as studios build one, scene by scene and layer by layer, in a form a simulator steps as written, and an accepted environment will show whether the approach holds.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[12] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[13] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[15] G. He, R. Luo, W.-C. Ma, and H. Averbuch-Elor, "Thinking in Blender: Staged executable inverse graphics with vision-language models," *arXiv preprint arXiv:2606.02580*, 2026.

[16] H. Wang *et al.*, "Code as worlds: Agentic discovery of executable world representations for physical reasoning," *arXiv preprint arXiv:2608.27549*, 2026.

[17] Y. Gao *et al.*, "3DCodeBench: Benchmarking agentic procedural 3D modeling via code," *arXiv preprint arXiv:2606.01057*, 2026.

[18] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[19] R. Shen *et al.*, "4DCodeBench: Benchmarking agents on inverse graphics of dynamic scenes," *arXiv preprint arXiv:2610.03715*, 2026.

[20] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[21] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[22] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[23] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[24] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[25] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[26] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[27] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *IEEE/RSJ international conference on intelligent robots and systems (IROS)*, 2012, pp. 5026–5033.

[28] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[29] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[30] Google DeepMind and NVIDIA, "MuJoCo Warp: A GPU-optimized version of the MuJoCo physics simulator." 2026. Available: https://github.com/google-deepmind/mujoco_warp

[31] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[32] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[33] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[34] E. Coumans and Y. Bai, "Bullet physics SDK and PyBullet." 2021. Available: https://github.com/bulletphysics/bullet3

[35] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[36] Y. Wu, K. Chen, T. Zhang, *et al.*, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," *arXiv preprint arXiv:2211.06687*, 2022.

[37] J. Saito, J. Li, M. de Ruyter, M. Guerrero, *et al.*, "SOMA: Unifying parametric human body models," *arXiv preprint arXiv:2603.16858*, 2026.

[38] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[39] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[40] S. D. Uhlrich *et al.*, "OpenCap: Human movement dynamics from smartphone videos," *PLOS Computational Biology*, vol. 19, no. 10, p. e1011462, 2023.

[41] V. Caggiano, H. Wang, G. Durandau, M. Sartori, and V. Kumar, "MyoSuite: A contact-rich simulation suite for musculoskeletal motor control," *arXiv preprint arXiv:2205.13600*, 2022.

[42] K. R. S. Holzbaur, W. M. Murray, and S. L. Delp, "A model of the upper extremity for simulating musculoskeletal surgery and analyzing neuromuscular control," *Annals of Biomedical Engineering*, vol. 33, no. 6, pp. 829–840, 2005, doi: 10.1007/s10439-005-3320-7.

[43] K. R. Saul *et al.*, "Benchmarking of dynamic simulation predictions in two software platforms using an upper limb musculoskeletal model," *Computer Methods in Biomechanics and Biomedical Engineering*, vol. 18, no. 13, pp. 1445–1458, 2015, doi: 10.1080/10255842.2014.916698.

[44] NVIDIA, "GEM-X: Monocular whole-body 3D human pose estimation using the SOMA body model." 2026. Available: https://github.com/NVlabs/GEM-X

[45] S. L. Delp *et al.*, "OpenSim: Open-source software to create and analyze dynamic simulations of movement," *IEEE Transactions on Biomedical Engineering*, vol. 54, no. 11, pp. 1940–1950, 2007.

[46] K. Werling *et al.*, "AddBiomechanics: Automating model scaling, inverse kinematics, and inverse dynamics from human motion data through sequential optimization," *PLOS ONE*, vol. 18, no. 11, p. e0295152, 2023.

[47] M. Stölzle, D. Müller, A. Serifi, R. Grandia, E. Knoop, and M. Bächer, "PhysibleMDM: Enforcing inference-time physical feasibility constraints on motion diffusion models via noise optimization," *arXiv preprint arXiv:2610.30500*, 2026.

[48] Y. Huo, H. Zhang, C. Song, H. Song, Z. Zuo, and T. Yu, "BioHuman: Learning biomechanical human representations from video," *arXiv preprint arXiv:2605.14772*, 2026.

[49] M. Simos, A. S. Chiappa, and A. Mathis, "KINESIS: Motion imitation for human musculoskeletal locomotion," *arXiv preprint arXiv:2503.14637*, 2025.

[50] C. Li *et al.*, "Towards embodied AI with MuscleMimic: Unlocking full-body musculoskeletal motor learning at scale," *arXiv preprint arXiv:2603.25544*, 2026.

[51] Z. Luo, J. Cao, A. Winkler, K. Kitani, and W. Xu, "Perpetual humanoid control for real-time simulated avatars," *arXiv preprint arXiv:2305.06456*, 2023.